TOPReward: Token Probabilities as Hidden Zero-Shot Rewards for Robotics

Shirui Chen[1,2] **Cole Harrison**[3] **Ying-Chun Lee**[1]

Angela Jin Yang[1] **Zhongzheng Ren**[1,2,4] **Lillian J. Ratliff**[1]

Jiafei Duan[†,1,2] **Dieter Fox**[†,1,2] **Ranjay Krishna**[†,1,2]

https://topreward.github.io/webpage/

Abstract: General-purpose robot learning requires dense, instruction-conditioned feedback that can distinguish meaningful task progress from stalled, failed, or partially completed behavior. Yet obtaining such feedback at scale remains difficult, since existing approaches often rely on manual progress annotations, task-specific demonstrations, or reward models trained on curated robot datasets. We introduce `TOPReward`, a training-free progress reward method that probes pretrained Video-Language Models (VLMs) through their internal token probabilities rather than asking them to generate numerical progress values. Given a video prefix and a language instruction, `TOPReward` measures the model's likelihood that the instructed task has been completed, converting latent video-language understanding into a dense reward signal without task-specific reward-model training or manually annotated progress labels. We evaluate `TOPReward` on ManiRewardBench, our real-world manipulation benchmark spanning 130 unique tasks and four robot platforms, as well as on Open X-Embodiment datasets. Across these settings, `TOPReward` substantially outperforms prior training-free VLM reward methods on open-source models and is competitive with a trained reward-model baseline on progress-estimation metrics, while requiring no reward-model training. Additional analyses show that the reward is sensitive to the specified instruction and is not explained by time index alone. Finally, `TOPReward` supports downstream applications including success detection and offline reward-weighted behavior cloning.

Keywords: robot learning, reward models, vision-language models

1 Introduction

Recent advances in Vision-Language-Action (VLA) models have spurred significant interest in leveraging Reinforcement Learning (RL) to achieve truly generalizable real-world performance Lei et al. [1], Chen et al. [2], Xiao et al. [3]. However, real-world RL remains bottlenecked by the extreme sample inefficiency inherent in sparse reward signals. To bridge this gap, the research community has pivoted toward developing generalizable process reward models that provide fine-grained and dense feedback. Current efforts typically focus on directly fine-tuning vision-language models (VLMs) as process reward functions on curated robot datasets Duan et al. [4], Budzianowski et al. [5], Ma et al. [6], Lin et al. [7] or training specific networks with custom-collected datasets [8, 2]. For instance, RoboDopamine [9] trains a reward model on 3,400+ hours of manipulation data with step-aware multi-view perception, but requires task-specific demonstrations for adaptation. Likewise, Lee et al. [10] introduces RoboReward, which fine-tunes a VLM on a large-scale set of robot trajectories with human-provided success labels and progress scores. However, these approaches rely on costly data

[†]Co-advised. [1]University of Washington [2]Allen Institute for AI [3]Amazon [4]University of North Carolina at Chapel Hill. Correspondence to: Shirui Chen <sc256@uw.edu>.

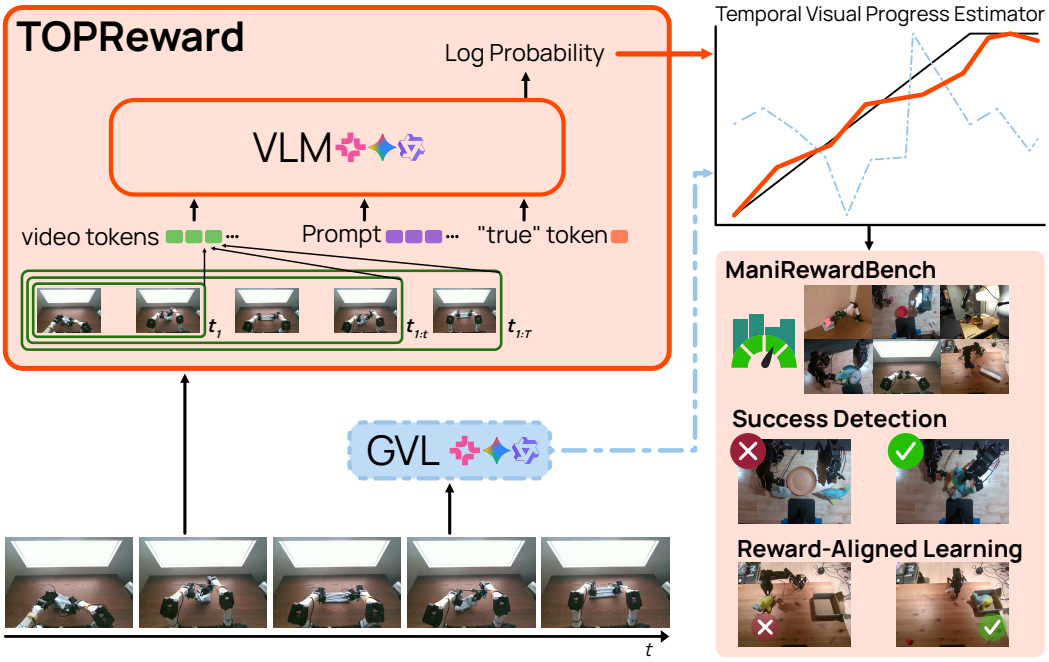


Figure 1: **Result highlights**. `TOPReward` enables effective zero-shot estimation of task progress across diverse and challenging real-world manipulation tasks, without task-specific training. Across several VLM backbones, `TOPReward` provides a temporally consistent visual reward signal that supports multiple downstream applications, including success detection, policy improvement, and evaluation on our in-house benchmark, `ManiRewardBench`.

requirements: RoboDopamine requires additional demonstrations when adapting to each new task, and RoboReward reports clear gaps across different embodiments and views, indicating limited generalization guarantees. Therefore, while these efforts demonstrate promise for narrow-domain reward models, they still rely on extensive data collection and struggle to generalize beyond the training distribution.

To circumvent the high costs of task-specific fine-tuning, we investigate the use of pretrained VLMs as zero-shot reward models. Our goal is to harness the pretrained visual-language representations embedded in these models to provide generalizable instruction-conditioned progress signals. Recent literature [11, 6, 12] has converged on progress estimation as a useful proxy for value, as it provides dense temporal feedback for learning and adaptation. The current state-of-the-art training-free method, GVL [6], casts progress estimation as visual question-answering—but performs well only on proprietary VLMs like Gemini and GPT-4 [5], collapsing on open-source alternatives. Indeed, contemporary studies [13] suggest that open-source VLMs are not yet "robotics-ready" for progress estimation.

In this work, we challenge the prevailing assumption that open-source VLMs are unfit for reward modeling by introducing a novel formulation for zero-shot progress estimation. We hypothesize that the failure of current open-source models stems not from a lack of temporal understanding, but from the representation bottleneck of textual output—specifically, the models' inconsistent instruction-following and their notorious bias in representing numerical tokens. To resolve this, we present `TOPReward`, a probabilistically grounded progress estimator that bypasses autoregressive text generation entirely. Instead of prompting the VLM to output the value of the completion percentage in text space, `TOPReward` extracts the model's internal "belief" by analyzing the probabilistic distribution of its token logits. By measuring the shift in confidence toward task-completion tokens over time, we derive a continuous, well-posed progress signal directly from the VLM's latent probabilistic representation. This approach requires zero additional training or fine-tuning, revealing that robust reward modeling is an emergent capability already present in pretrained video VLMs—if one looks beyond the text.

To enable rigorous evaluation of progress estimation methods, we introduce `ManiRewardBench`, a benchmark comprising 130 unique real-world manipulation tasks spanning multiple robot platforms (Franka Emika arms, Single-Arm/Bimanual YAM, SO-100/101) with temporal annotations of task progress. We demonstrate that `TOPReward` can effectively track task progress across this diverse benchmark. The raw completion probability supports cross-trajectory success detection, while normalized prefix scores provide within-trajectory progress curves for evaluation and visualization. We further use `TOPReward` to weight expert examples in offline behavior cloning. In real-world deployment on six single-arm SO-100 manipulation tasks, reward-weighted fine-tuning with `TOPReward` consistently improves success rates over standard behavior cloning, achieving up to 10 out of 10 successes on challenging tasks where baseline behavior cloning reaches only 7 out of 10.

2 Related Work

The reward bottleneck for VLA. Large-scale vision-language-action policies such as OpenVLA [14], π_0 [15], MolmoAct [16] and Gemini Robotics [17] have demonstrated strong language-conditioned manipulation capabilities across diverse embodiments, yet reliably deploying them in real-world settings remains an open problem [18]. A natural path forward is reinforcement learning with online or offline fine-tuning, but RL in practice hinges on the availability of a reward signal—one that is traditionally hand-crafted per task in robotics, difficult to scale, and brittle under distribution shift [19, 20]. Recent efforts have applied RL to improve generalist robot policies in real-world deployment [21, 22, 23, 24, 25, 26, 27], including RL-100 [1], which trains diffusion-based visuomotor policies directly on real robots using human-provided success signals, and $\pi^*_{0.6}$ [28], which improves π_0 through real-world RL with human-annotated episode outcomes. All of these approaches, however, remain reliant on manual reward specification, underscoring the need for automated, scalable alternatives.

Learned reward models. A long line of work seeks to replace hand-crafted rewards with learned alternatives [29, 30, 31]. Embedding-based methods such as VIP [32], LIV [33], and R3M [34] learn visual representations that capture progress toward a goal, but require task-specific fine-tuning and offer limited language grounding. VQA-style approaches such as SuccessVQA [35] and related frameworks [36, 37] reframe reward as a binary classification problem—asking a VLM whether the task succeeded—but produce signals too coarse for dense reward shaping [38, 39]. Code-generation strategies like Eureka [40] synthesize reward functions via LLMs, yet depend on access to simulation ground truth. More recently, generalist reward models such as RoboReward [10] and RoboDopamine [9] are trained on large-scale datasets of successes and failures to predict progress scores or distance-to-goal estimates [41, 42]. While these models move toward broader coverage, they still require domain-specific training data and can struggle to generalize across embodiments and environments [43]. A common thread across all these approaches is the dependence on training or domain-specific resources—a requirement our method avoids.

Training-free value estimation with VLMs. A separate and more relevant line of work asks whether VLMs can estimate task progress without any additional training. Generative Value Learning (GVL) [6] poses progress prediction as a temporal ordering problem: given a batch of shuffled trajectory frames, the VLM is prompted to assign per-frame progress scores, exploiting its semantic grounding to rank frames by task completion. This enables various downstream applications such as dataset filtering and advantage-weighted regression without task-specific reward engineering. The OpenGVL benchmark [5] evaluates this paradigm across diverse tasks and model families, revealing that open-source VLMs fall substantially short of their proprietary counterparts on temporal progress prediction. In this paper, we hypothesize that the reason for this is not a lack of visual understanding in open-source VLMs but rather the instability of numeric token generation—LLMs are poorly calibrated when asked to produce precise numerical outputs [44, 45]. This observation motivates a fundamental shift: rather than asking a VLM to *generate* a progress value, one can instead probe what the model already *knows* through its internal representations.

Internal representations as reward signals. A growing body of work in NLP has shown that a model's internal activations—logits, hidden states, and embeddings—track its certainty and factual

accuracy more reliably than its generated text [46, 47, 48, 49, 50]. In robotics, recent methods have begun to leverage such representations for reward definition [11, 51], bypassing the instabilities of text generation. Our work, `TOPReward`, takes this principle further: instead of prompting a VLM to generate numeric progress estimates, we pose a binary completion query ("does this trajectory complete the task?") and extract the probability of the affirmative token as a continuous reward signal. This formulation is zero-shot, requires no fine-tuning or domain-specific data, and yields a useful progress signal that scales to the 130-task `ManiRewardBench` benchmark as well as the Open X-Embodiment dataset across multiple robot platforms.

3 `TOPReward`

The stark difference between GVL's [6] performance on Gemini versus open-source models might mislead one into believing that only the most powerful VLM can accurately estimate the progress of a robotic trajectory. Indeed, for a model to output well-formatted and accurate progress estimates for all shuffled frames, it needs strong instruction-following capability and an accurate internal representation of numerical values. However, neither of these is necessarily correlated with the model's underlying video understanding capability. Therefore, we propose `TOPReward`, a method that leverages the internal understanding of the VLM to produce a progress estimator without requiring accurate, well-formatted numerical generation.

Problem setup. We formulate the progress estimation task as follows: given an instruction x and a video trajectory $\tau_{1:T} = (I_1, \ldots, I_T)$ (frames in chronological order), our goal is to produce a scalar prefix score for each $\tau_{1:t}$ that reflects accumulated evidence that the instruction has been completed.

3.1 Token probability as the reward

Key idea. We use the VLM's *internal output, i.e., predicted token probabilities* as the reward. Concretely, we ask the model to judge whether the observed trajectory *completes* the instruction and score the probability of an affirmative answer (e.g. the token `True`).

Let p_θ be a VLM defining a next-token distribution with pretrained weights θ. We form a prompt u that grounds the judgment in the video:

> "`<|video|>` The above video shows a robot manipulation trajectory that completes the following task: {`INSTRUCTION`}. Decide whether the above statement is True or not. The answer is: $\{a\}$"

and compute the probability of the answer token sequence a="True" [0]. We choose `True` because we found boolean tokens to show the clearest success–failure separation, with `True` exhibiting the largest absolute difference in mean token probability across episodes (see Section B for the token-probability comparison). Denoting the (video-conditioned) textual context by $c(\tau_{1:t}, u)$, we define the reward for a prefix to be

$$r_t \;=\; \log p_\theta(a \mid c(\tau_{1:t}, u))\,. \tag{1}$$

In this way, we construct a conditional completion score for the trajectory-instruction pair while sidestepping the need for the language model to generate calibrated numerical values. The raw score r_t is causal because it depends only on the prefix $\tau_{1:t}$ and the instruction. As we will see in Section 4, r_t tends to increase on successful demonstrations as visual evidence accumulates, while its absolute level is useful for comparing complete successful and failed trajectories.

Chat templates. For open-source `TOPReward` experiments, we score the prompt directly rather than adding a generation-oriented chat template. Our ablation in Section E.2 shows that chat templates can substantially reduce performance, so we use direct prompt scoring whenever the model interface permits it. The Gemini API enforces chat-style formatting, which may partly explain its weaker `TOPReward` results in Table 1.

[0] We also explore another variant that evaluates the probability over the entire instruction which consists of multiple tokens, but found it to be less effective. See Section A for details.

3.2 Progress estimation from trajectory prefixes

Prefix sampling. To obtain a temporal progress curve, we evaluate Eq. (1) on a set of K uniformly spaced prefix lengths $\{t_k\}_{k=1}^{K}$ with $1 = t_1 < \cdots < t_K = T$. This involves K model forwards and produces rewards $\{r_{t_k}\}$ summarizing how completion evidence accumulates over time (Figure 8).

Normalization. The raw log-probability range is $(-\infty, 0]$, while VOC and visualizations are easier to interpret on a bounded scale. For within-trajectory evaluation, we therefore use min-max normalization to map rewards to a normalized progress score s_{t_k} within each episode:

$$s_{t_k} \;=\; \frac{r_{t_k} - \min_j r_{t_j}}{\max_j r_{t_j} - \min_j r_{t_j} + \varepsilon}, \tag{2}$$

with a small ε for numerical stability. This normalization is non-causal because the episode-level minimum and maximum are known only after all sampled prefixes are scored. We use it for within-trajectory progress visualization and VOC evaluation, not as the raw online reward signal.

Positive weights for downstream offline learning. When a per-step weight is needed for reward-weighted offline behavior cloning, we use the `TOPReward`increment to construct a positive weighting signal:

$$\Delta_{t_k} \;=\; \min\left\{\exp(\beta \cdot (r_{t_k} - r_{t_{k-1}})), \delta_{\max}\right\}, \tag{3}$$

where β controls the sharpness of the weighting and $\delta_{\max}$ caps large weights for training stability. The weight is always positive: steps with decreasing `TOPReward` score are downweighted rather than treated as negative supervision, which keeps the flow-matching objective well-posed.

4 Experiments

We evaluate `TOPReward` across three main dimensions: (1) zero-shot progress estimation on large-scale robot datasets using VOC and complementary progress metrics (Section 4.1); (2) instruction sensitivity; and (3) downstream applications including success detection and real-world reward-weighted behavior cloning.

VLM backbones. We evaluate `TOPReward` on Qwen3-VL-8B and Qwen3-VL-32B [52], Molmo2-8B [53], and Gemini-2.5-Pro [54]. Qwen3-VL and Molmo2 are open-source video-language models; Gemini-2.5-Pro serves as a proprietary baseline with logit access.

Benchmark. `ManiRewardBench` contains 130 unique real-world manipulation tasks collected across four robot platforms: Franka, SO-100/101, single-arm YAM, and bimanual YAM. Episodes are annotated with subtask boundaries for stage-aware progress evaluation, and a separate 23-task failure split supports success detection. Additional dataset details are in Section F.

4.1 Large-scale real-world evaluation

To evaluate zero-shot progress estimation, we test whether `TOPReward`produces accurate dense progress estimates on expert robotic trajectories from `ManiRewardBench` and Open X-Embodiment (OXE). We mainly compare against GVL [5, 6], the state-of-the-art training-free progress estimator, which prompts a VLM to assign numerical progress values to shuffled frames.

Metrics. Following standard practice [6, 33], we use Value-Order Correlation (VOC) to measure Spearman's rank correlation between the chronological order of input video frames and the predicted values,

$$\text{VOC} = \text{rank-correlation}\big(\text{argsort}(s_{t_1}, s_{t_2}, \cdots, s_{t_K}), (t_1, t_2, \cdots, t_K)\big). \tag{4}$$

VOC ranges from -1 to 1. When VOC equals -1, the predicted order is exactly the opposite of the ground truth, and VOC $= 1$ indicates perfect alignment. Because VOC measures only temporal ordering, it should not be interpreted as a standalone test of instruction grounding. We therefore report complementary progress metrics (Kendall tau-b, Pearson correlation, and MAE) against the available completion annotations.

Table 1: **Results on the filtered Open X-Embodiment dataset.** VOC reports mean dataset-level VOC over the same 33 filtered datasets and 20 episodes per dataset. Higher is better except MAE.

Method	Model	VOC	Kendall	Pearson	MAE
GVL	Molmo2-8B	-0.019	0.000	0.001	0.500
GVL	Qwen3-VL-8B	0.218	0.192	0.443	0.396
GVL	Gemini-2.5-Pro	0.572	0.534	0.669	0.218
TOPReward	Molmo2-8B	0.525	0.427	0.503	0.308
TOPReward	Qwen3-VL-8B	0.874	0.771	0.856	**0.163**
TOPReward	Qwen3-VL-32B	**0.890**	**0.799**	**0.870**	0.191
TOPReward	Gemini-2.5-Pro	0.430	0.332	0.399	0.321
Robometer	4B	0.838	0.765	0.840	0.180

Table 2: **Results on** ManiRewardBench**.** We report metrics averaged over the 497 successful-evaluation episodes. This progress-evaluation subset contains 113 tasks across four robot platforms: LeRobot, Franka, Bimanual YAM, and Single-arm YAM. VOC and Kendall tau-b measure rank agreement, Pearson measures linear agreement, and MAE measures normalized progress error. Higher is better except MAE.

Method	Model	VOC	Kendall	Pearson	MAE
GVL	Molmo2-8B	-0.003	-0.002	-0.003	0.514
GVL	Qwen3-VL-8B	0.316	0.261	0.326	0.392
TOPReward	Molmo2-8B	0.619	0.484	0.569	0.316
TOPReward	Qwen3-VL-8B	**0.942**	**0.858**	**0.915**	**0.126**
TOPReward	Qwen3-VL-32B	0.864	0.771	0.824	0.194
Robometer	4B	0.858	0.768	0.861	0.160

Results on Open X-Embodiment. The Open X-Embodiment (OXE) dataset [55] is a collection of 50 academic robot datasets spanning diverse tasks, camera configurations, and robot platforms. We select 33 high-quality OXE datasets from the LeRobot collection and use this same filtered set for every row in Table 1. On OXE datasets, TOPReward substantially outperforms GVL on open-source models such as Qwen3-VL and Molmo. Complementary progress metrics show the same trend beyond VOC across available rows. We also compare against the trained Robometer-4B progress head on the same filtered OXE split using 15 uniformly sampled frames per episode, and find that TOPRewardis slightly better. On the proprietary Gemini-2.5-Pro, GVL performs better (0.572) while TOPReward achieves 0.430, reflecting the chat-template issue discussed in Section E.2.

Results on ManiRewardBench**.** We evaluate progress estimation on the successful-trajectory subset of ManiRewardBench, covering 113 tasks and 497 episodes across 4 robotic platforms. Results are shown in Table 2. On Qwen3-VL-8B, TOPReward achieves the strongest aggregate progress metrics, substantially outperforming GVL on the same backbone. On Molmo2-8B, GVL is near zero across rank and correlation metrics, while TOPReward recovers a useful progress signal with positive VOC, Kendall tau-b, and Pearson correlation. Compared with the trained Robometer-4B baseline, TOPReward is competitive across rank, correlation, and error metrics while requiring no reward-model training. Detailed per-task breakdowns and distribution plots are in Section D.

Qualitative results. We visualize representative progress traces in Figure 7. The traces demonstrate that TOPReward produces smooth, monotonically increasing progress signals that closely track stage-aware ground-truth task completion (computed from annotated subtask boundaries) across diverse manipulation tasks. In contrast, Gemini-GVL exhibits noisier predictions with frequent non-monotonic fluctuations. Notably, TOPReward correctly captures the temporal structure of multi-step tasks, with progress plateaus corresponding to intermediate subtask completions and accelerations during active manipulation phases.

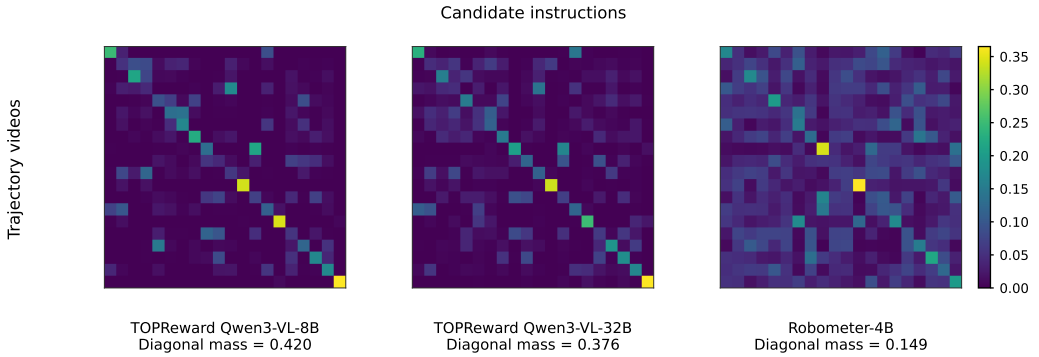


Figure 2: **Instruction sensitivity.** Instruction-grounding matrices for TOPReward and Robometer-4B. Rows are terminal trajectory videos and columns are candidate task instructions. For TOPReward, each cell is the terminal TOPReward score for that video–instruction pair; for Robometer-4B, each cell is its terminal reward prediction. We apply Sinkhorn normalization for visualization and report mean diagonal mass, so both rows and columns sum to one and bright diagonal cells indicate strong one-to-one matching between trajectories and their intended instructions.

4.2 Instruction sensitivity

Order-based metrics alone cannot establish that a reward is grounded in the language instruction: an instruction-agnostic ordering score can be highly correlated with progress on successful demonstrations while assigning the same values for every instruction. We therefore evaluate whether final-trajectory rewards discriminate the matched instruction from plausible but incorrect task descriptions. For each video, we score the same terminal trajectory under each candidate instruction and form a video-task $\times$ instruction-task matrix, shown in Figure 2. For TOPReward, each raw entry is the terminal TOPReward score for a video–instruction pair; for Robometer-4B, each raw entry is its terminal reward prediction. We convert these score matrices to positive weights and apply Sinkhorn normalization, so each row and each column sums to one. Strong diagonal structure and high mean diagonal mass in this doubly normalized matrix indicate one-to-one matching between trajectories and their intended instructions rather than instruction-agnostic temporal ordering.

Figure 2 summarizes instruction disambiguation using mean diagonal mass after Sinkhorn normalization of the displayed score matrix. This gives a bidirectional one-to-one matching view: each trajectory distributes mass across candidate instructions, and each instruction distributes mass across candidate trajectories. The random baseline is $1/20 = 0.05$ on this 20-task split. The diagonal mass is 0.420 for `TOPReward` Qwen3-VL-8B, 0.376 for `TOPReward` Qwen3-VL-32B, and 0.149 for Robometer-4B, showing that TOPReward produces substantially stronger matched video–instruction structure than the trained reward-model baseline. As a scalar control on the 150 successful LeRobot episodes in the progress-evaluation subset, pairing each video with its correct instruction gives mean raw reward -1.68, while pairing the same videos with mismatched instructions from other tasks drops the mean raw reward to -5.04. Thus raw instruction-conditioned rewards and confusion matrices provide the evidence for semantic grounding beyond temporal ordering.

4.3 Success detection

Setup and results. We evaluate on the cleaned failure trajectory split of `ManiRewardBench`, which contains 150 successful and 129 failed LeRobot attempts. Taking inspiration from self-consistency prompting [56], we combine multiple TOPReward scores using different prompts to obtain a success score for success detection as defined in Section C.

As shown in Table 3, Qwen3-VL-32B reaches 0.965 ROC-AUC and Qwen3-VL-8B reaches 0.939 ROC-AUC, both exceeding the trained Robometer-4B success head without reward-model training or success-label supervision.

Table 3: **Success detection results.** We evaluate 279 cleaned LeRobot episodes from `ManiReward Bench` (150 successful, 129 failed). `TOPReward` uses the training-free prefix-margin plus full-video Yes/No score defined in Section C; GVL uses per-trajectory VOC; Robometer-4B uses its trained success head.

Method	Score	ROC-AUC	PR-AUC
GVL (Qwen3-VL-8B)	VOC	0.728	0.893
`TOPReward` (Qwen3-VL-8B)	success score	0.939	0.952
`TOPReward` (Qwen3-VL-32B)	success score	**0.965**	**0.973**
Robometer-4B	trained success head	0.930	0.951

Table 4: **Real-world experiments.** Partial success score out of 10 trials for reward-weighted behavior cloning on single-arm SO-100 tasks.

Task	Pretrained	BC	TOP-RWBC (Ours)
Place toy car in box	1	2	**3**
Stack red cube on green cube	1.33	1	**2.33**
Put pen into cup	1.67	5.67	**6.33**
Place doll in box	0	7	**10**
Pick up cube	4	7	**10**
Put cube in cup	4	6	**9**

4.4 Real-world reward-weighted behavior cloning

To further showcase `TOPReward` as a signal for policy improvement, we use it to derive positive weights for an offline flow-matching behavior cloning objective. We start from a base policy π_0 pretrained on 200 hours of the publicly available single-arm SO-100 dataset (HuggingFace). For each of six real-world tasks, we collect 50 demonstrations (potentially noisy and suboptimal) and use `TOPReward` to compute prefix scores for the demonstrations. We convert progress increments to weights using Equation (3), then fine-tune π_0 with

$$\mathcal{L}_{\text{RWBC}} = \mathbb{E}_{p(a|o),q(a_t|a)}\left[\Delta_t \cdot \|v_\theta(a_t, t \mid o) - (a - \epsilon)\|^2\right], \tag{5}$$

where $a_t = (1-t)\epsilon + ta$, $t \sim \mathcal{U}(0,1)$, $\epsilon \sim \mathcal{N}(0, I)$, and Δ_t is the `TOPReward`-derived positive weight computed with $\beta = 0.2$ and $\delta_{\max} = 2.0$ in Equation (3). We compare TOP-RWBC against behavior cloning (BC) on the same data, measuring partial success as the fraction of predefined subtasks completed per trial, summed over 10 trials. As shown in Table 4, TOP-RWBC improves over BC on all six tasks.

5 Limitations

`TOPReward` inherits the visual perception limitations of the underlying VLM: tasks requiring fine-grained spatial reasoning, precise alignment, or small-object manipulation may receive noisy progress estimates when the model cannot visually distinguish intermediate states. The raw prefix score in Equation (1) is causal, and the per-episode min-max normalization in Equation (2) is non-causal and is used for within-trajectory evaluation and visualization. Finally, `TOPReward` is sensitive to chat template formatting and answer-token choice, so deployments should validate the prompt and tokenizer behavior for the chosen VLM backbone.

6 Conclusion

We presented `TOPReward`, a zero-shot progress reward method that repurposes token probabilities of pretrained video VLMs as instruction-conditioned progress signals for robotic manipulation. By querying the model's internal belief about instruction completion rather than requiring it to generate calibrated numerical outputs, `TOPReward` sidesteps the well-known limitations of VLMs in numerical reasoning and instruction following. Across Open X-Embodiment and our newly introduced `Mani`

RewardBench benchmark, TOPReward substantially outperforms GVL on open-source models. It also remains competitive with robotics reward models trained on large-scale data across progress-estimation metrics, while requiring no reward-model training. The instruction-disambiguation matrix and wrong-instruction raw-reward drop further show that rewards depend on the language instruction rather than prefix position alone. Finally, TOPRewardsupports success detection without reward-model training, and TOPReward can be used for reward-weighted behavior cloning, yielding consistent improvements over standard BC across six real-world SO-100 manipulation tasks.

References

[1] K. Lei, H. Li, D. Yu, Z. Wei, L. Guo, Z. Jiang, Z. Wang, S. Liang, and H. Xu. Rl-100: Performant robotic manipulation with real-world reinforcement learning, 2025. URL https://arxiv.org/abs/2510.14830.

[2] Q. Chen, J. Yu, M. Schwager, P. Abbeel, Y. Shentu, and P. Wu. Sarm: Stage-aware reward modeling for long horizon robot manipulation. *arXiv preprint arXiv:2509.25358*, 2025.

[3] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie, F. Hu, J. Wu, Z. Luo, L. Fan, et al. Self-improving vision-language-action models with data generation via residual rl. *arXiv preprint arXiv:2511.00091*, 2025.

[4] J. Duan, W. Pumacay, N. Kumar, Y. R. Wang, S. Tian, W. Yuan, R. Krishna, D. Fox, A. Mandlekar, and Y. Guo. Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation. *arXiv preprint arXiv:2410.00371*, 2024.

[5] P. Budzianowski, E. Wiśnios, G. Góral, I. Kulakov, V. Petrenko, and K. Walas. Opengvl–benchmarking visual temporal progress for data curation. *arXiv preprint arXiv:2509.17321*, 2025.

[6] Y. J. Ma, J. Hejna, C. Fu, D. Shah, J. Liang, Z. Xu, S. Kirmani, P. Xu, D. Driess, T. Xiao, et al. Vision language models are in-context value learners. In *The Thirteenth International Conference on Learning Representations*, 2024.

[7] Z. Lin, J. Duan, H. Fang, D. Fox, R. Krishna, C. Tan, and B. Wen. Failsafe: Reasoning and recovery from failures in vision-language-action models. *arXiv preprint arXiv:2510.01642*, 2025.

[8] J. Zhang, Y. Luo, A. Anwar, S. A. Sontakke, J. J. Lim, J. Thomason, E. Biyik, and J. Zhang. Rewind: Language-guided rewards teach robot policies without new demonstrations. *arXiv preprint arXiv:2505.10911*, 2025.

[9] H. Tan, S. Chen, Y. Xu, Z. Wang, Y. Ji, C. Chi, Y. Lyu, Z. Zhao, X. Chen, P. Co, et al. Robodopamine: General process reward modeling for high-precision robotic manipulation. *arXiv preprint arXiv:2512.23703*, 2025.

[10] T. Lee, A. Wagenmaker, K. Pertsch, P. Liang, S. Levine, and C. Finn. Roboreward: General-purpose vision-language reward models for robotics. *arXiv preprint arXiv:2601.00675*, 2026.

[11] J. Rocamonde, V. Montesinos, E. Nava, E. Perez, and D. Lindner. Vision-language models are zero-shot reward models for reinforcement learning. *arXiv preprint arXiv:2310.12921*, 2023.

[12] K. Baumli, S. Baveja, F. Behbahani, H. Chan, G. Comanici, S. Flennerhag, M. Gazeau, K. Holsheimer, D. Horgan, M. Laskin, et al. Vision-language models as a source of rewards. *arXiv preprint arXiv:2312.09187*, 2023.

[13] J. Zhang, C. Qian, H. Sun, H. Lu, D. Wang, L. Xue, and H. Liu. Progresslm: Towards progress reasoning in vision-language models. *arXiv preprint arXiv:2601.15224*, 2026.

[14] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. Openvla: An open-source vision-language-action model, 2024. URL https://arxiv.org/abs/2406.09246.

[15] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0: A vision-language-action flow model for general robot control, 2026. URL https://arxiv.org/abs/2410.24164.

[16] J. Lee, J. Duan, H. Fang, Y. Deng, S. Liu, B. Li, B. Fang, J. Zhang, Y. R. Wang, S. Lee, et al. Molmoact: Action reasoning models that can reason in space. *arXiv preprint arXiv:2508.07917*, 2025.

[17] G. R. Team, S. Abeyruwan, J. Ainslie, J.-B. Alayrac, M. G. Arenas, T. Armstrong, A. Balakrishna, R. Baruch, M. Bauza, M. Blokzijl, S. Bohez, K. Bousmalis, A. Brohan, T. Buschmann, A. Byravan, S. Cabi, K. Caluwaerts, F. Casarini, O. Chang, J. E. Chen, X. Chen, H.-T. L. Chiang, K. Choromanski, D. D'Ambrosio, S. Dasari, T. Davchev, C. Devin, N. D. Palo, T. Ding, A. Dostmohamed, D. Driess, Y. Du, D. Dwibedi, M. Elabd, C. Fantacci, C. Fong, E. Frey, C. Fu, M. Giustina, K. Gopalakrishnan, L. Graesser, L. Hasenclever, N. Heess, B. Hernaez, A. Herzog, R. A. Hofer, J. Humplik, A. Iscen, M. G. Jacob, D. Jain, R. Julian, D. Kalashnikov, M. E. Karagozler, S. Karp, C. Kew, J. Kirkland, S. Kirmani, Y. Kuang, T. Lampe, A. Laurens, I. Leal, A. X. Lee, T.-W. E. Lee, J. Liang, Y. Lin, S. Maddineni, A. Majumdar, A. H. Michaely, R. Moreno, M. Neunert, F. Nori, C. Parada, E. Parisotto, P. Pastor, A. Pooley, K. Rao, K. Reymann, D. Sadigh, S. Saliceti, P. Sanketi, P. Sermanet, D. Shah, M. Sharma, K. Shea, C. Shu, V. Sindhwani, S. Singh, R. Soricut, J. T. Springenberg, R. Sterneck, R. Surdulescu, J. Tan, J. Tompson, V. Vanhoucke, J. Varley, G. Vesom, G. Vezzani, O. Vinyals, A. Wahid, S. Welker, P. Wohlhart, F. Xia, T. Xiao, A. Xie, J. Xie, P. Xu, S. Xu, Y. Xu, Z. Xu, Y. Yang, R. Yao, S. Yaroshenko, W. Yu, W. Yuan, J. Zhang, T. Zhang, A. Zhou, and Y. Zhou. Gemini robotics: Bringing ai into the physical world, 2025. URL https://arxiv.org/abs/2503.20020.

[18] R. Firoozi, J. Tucker, S. Tian, A. Majumdar, J. Sun, W. Liu, Y. Zhu, S. Song, A. Kapoor, K. Hausman, et al. Foundation models in robotics: Applications, challenges, and the future. *The International Journal of Robotics Research*, 44(5):701–739, 2025.

[19] J. Kober, J. A. Bagnell, and J. Peters. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11):1238–1274, 2013.

[20] G. Dulac-Arnold, D. Mankowitz, and T. Hester. Challenges of real-world reinforcement learning. *arXiv preprint arXiv:1904.12901*, 2019.

[21] J. Zhang, M. Heo, Z. Liu, E. Biyik, J. J. Lim, Y. Liu, and R. Fakoor. Extract: Efficient policy learning by extracting transferable robot skills from offline data. *arXiv preprint arXiv:2406.17768*, 2024.

[22] M. Nakamoto, S. Zhai, A. Singh, M. Sobol Mark, Y. Ma, C. Finn, A. Kumar, and S. Levine. Cal-ql: Calibrated offline rl pre-training for efficient online fine-tuning. *Advances in Neural Information Processing Systems*, 36:62244–62269, 2023.

[23] Y. Chen, S. Tian, S. Liu, Y. Zhou, H. Li, and D. Zhao. Conrft: A reinforced fine-tuning method for vla models via consistency policy. *arXiv preprint arXiv:2502.05450*, 2025.

[24] J. Hu, R. Hendrix, A. Farhadi, A. Kembhavi, R. Martín-Martín, P. Stone, K.-H. Zeng, and K. Ehsani. Flare: Achieving masterful and adaptive robot policies with large-scale reinforcement learning fine-tuning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3617–3624. IEEE, 2025.

[25] L. Ankile, A. Simeonov, I. Shenfeld, M. Torne, and P. Agrawal. From imitation to refinement-residual rl for precise assembly. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 01–08. IEEE, 2025.

[26] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine. Steering your diffusion policy with latent space reinforcement learning. *arXiv preprint arXiv:2506.15799*, 2025.

[27] P. Dong, S. Mirchandani, D. Sadigh, and C. Finn. What matters for batch online reinforcement learning in robotics? *arXiv preprint arXiv:2505.08078*, 2025.

[28] P. Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia, J. DiCarlo, D. Driess, M. Equi, A. Esmail, Y. Fang, C. Finn, C. Glossop, T. Godden, I. Goryachev, L. Groom, H. Hancock, K. Hausman, G. Hussein, B. Ichter, S. Jakubczak, R. Jen, T. Jones, B. Katz, L. Ke, C. Kuchi, M. Lamb, D. LeBlanc, S. Levine, A. Li-Bell, Y. Lu, V. Mano, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, C. Sharma, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, W. Stoeckle, A. Swerdlow, J. Tanner, M. Torne, Q. Vuong, A. Walling, H. Wang, B. Williams, S. Yoo, L. Yu, U. Zhilinsky, and Z. Zhou. $\pi^*_{0.6}$: a vla that learns from experience, 2025. URL https://arxiv.org/abs/2511.14759.

[29] D. A. Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural computation*, 3(1):88–97, 1991.

[30] J. Ho and S. Ermon. Generative adversarial imitation learning. *Advances in neural information processing systems*, 29, 2016.

[31] T. Hester, M. Vecerík, O. Pietquin, M. Lanctot, T. Schaul, B. Piot, A. Sendonaris, G. Dulac-Arnold, I. Osband, J. P. Agapiou, J. Z. Leibo, and A. Gruslys. Learning from demonstrations for real world reinforcement learning. *ArXiv*, abs/1704.03732, 2017. URL https://api.semanticscholar.org/CorpusID:15254659.

[32] Y. J. Ma, S. Sodhani, D. Jayaraman, O. Bastani, V. Kumar, and A. Zhang. Vip: Towards universal visual reward and representation via value-implicit pre-training. *arXiv preprint arXiv:2210.00030*, 2022.

[33] Y. J. Ma, V. Kumar, A. Zhang, O. Bastani, and D. Jayaraman. Liv: Language-image representations and rewards for robotic control. In *International Conference on Machine Learning*, pages 23301–23320. PMLR, 2023.

[34] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta. R3m: A universal visual representation for robot manipulation. *arXiv preprint arXiv:2203.12601*, 2022.

[35] Y. Du, K. Konyushkova, M. Denil, A. Raju, J. Landon, F. Hill, N. De Freitas, and S. Cabi. Vision-language models as success detectors. *arXiv preprint arXiv:2303.07280*, 2023.

[36] A. Stone, T. Xiao, Y. Lu, K. Gopalakrishnan, K.-H. Lee, Q. Vuong, P. Wohlhart, S. Kirmani, B. Zitkovich, F. Xia, et al. Open-world object manipulation using pre-trained vision-language models. *arXiv preprint arXiv:2303.00905*, 2023.

[37] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022.

[38] C. Lynch, M. Khansari, T. Xiao, V. Kumar, J. Tompson, S. Levine, and P. Sermanet. Learning latent plans from play. In *Conference on robot learning*, pages 1113–1132. Pmlr, 2020.

[39] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, O. Pieter Abbeel, and W. Zaremba. Hindsight experience replay. *Advances in neural information processing systems*, 30, 2017.

[40] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv:2310.12931*, 2023.

[41] H. Wu, Y. Jing, C. Cheang, G. Chen, J. Xu, X. Li, M. Liu, H. Li, and T. Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. *arXiv preprint arXiv:2312.13139*, 2023.

[42] L. Fan, G. Wang, Y. Jiang, A. Mandlekar, Y. Yang, H. Zhu, A. Tang, D.-A. Huang, Y. Zhu, and A. Anandkumar. Minedojo: Building open-ended embodied agents with internet-scale knowledge. *Advances in Neural Information Processing Systems*, 35:18343–18362, 2022.

[43] D. Kalashnikov, J. Varley, Y. Chebotar, B. Swanson, R. Jonschkowski, C. Finn, S. Levine, and K. Hausman. Mt-opt: Continuous multi-task robotic reinforcement learning at scale. *arXiv preprint arXiv:2104.08212*, 2021.

[44] E. Wallace, Y. Wang, S. Li, S. Singh, and M. Gardner. Do nlp models know numbers? probing numeracy in embeddings. *arXiv preprint arXiv:1909.07940*, 2019.

[45] Z. Yuan, H. Yuan, C. Tan, W. Wang, and S. Huang. How well do large language models perform in arithmetic tasks? *arXiv preprint arXiv:2304.02015*, 2023.

[46] S. Kadavath, T. Conerly, A. Askell, T. Henighan, D. Drain, E. Perez, N. Schiefer, Z. Hatfield-Dodds, N. DasSarma, E. Tran-Johnson, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.

[47] K. Tian, E. Mitchell, A. Zhou, A. Sharma, R. Rafailov, H. Yao, C. Finn, and C. D. Manning. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback. *arXiv preprint arXiv:2305.14975*, 2023.

[48] A. Azaria and T. Mitchell. The internal state of an llm knows when it's lying. *arXiv preprint arXiv:2304.13734*, 2023.

[49] C. Burns, H. Ye, D. Klein, and J. Steinhardt. Discovering latent knowledge in language models without supervision. *arXiv preprint arXiv:2212.03827*, 2022.

[50] K. Liu, S. Casper, D. Hadfield-Menell, and J. Andreas. Cognitive dissonance: Why do language model outputs disagree with internal representations of truthfulness? *arXiv preprint arXiv:2312.03729*, 2023.

[51] C. Grislain, H. Rahimi, O. Sigaud, and M. Chetouani. I-failsense: Towards general robotic failure detection with vision-language models. *arXiv preprint arXiv:2509.16072*, 2025.

[52] S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, Z. Cheng, L. Deng, W. Ding, C. Gao, C. Ge, W. Ge, Z. Guo, Q. Huang, J. Huang, F. Huang, B. Hui, S. Jiang, Z. Li, M. Li, M. Li, K. Li, Z. Lin, J. Lin, X. Liu, J. Liu, C. Liu, Y. Liu, D. Liu, S. Liu, D. Lu, R. Luo, C. Lv, R. Men, L. Meng, X. Ren, X. Ren, S. Song, Y. Sun, J. Tang, J. Tu, J. Wan, P. Wang, P. Wang, Q. Wang, Y. Wang, T. Xie, Y. Xu, H. Xu, J. Xu, Z. Yang, M. Yang, J. Yang, A. Yang, B. Yu, F. Zhang, H. Zhang, X. Zhang, B. Zheng, H. Zhong, J. Zhou, F. Zhou, J. Zhou, Y. Zhu, and K. Zhu. Qwen3-vl technical report, 2025. URL https://arxiv.org/abs/2511.21631.

[53] C. Clark, J. Zhang, Z. Ma, J. S. Park, M. Salehi, R. Tripathi, S. Lee, Z. Ren, C. D. Kim, Y. Yang, V. Shao, Y. Yang, W. Huang, Z. Gao, T. Anderson, J. Zhang, J. Jain, G. Stoica, W. Han, A. Farhadi, and R. Krishna. Molmo2: Open weights and data for vision-language models with video understanding and grounding, 2026. URL https://arxiv.org/abs/2601.10611.

[54] G. C. et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities, 2025. URL https://arxiv.org/abs/2507.06261.

[55] A. O'Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE, 2024.

[56] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models. In *ICLR 2023*, 2023. URL https://arxiv.org/abs/2203.11171.

A Alternative Reward Formulation

In addition to the main formulation of `TOPReward` presented in Section 3, we also experimented with an alternative reward formulation that evaluates the probability of generating the entire instruction given the video trajectory. Specifically, we construct the following prompt,

> "`<|video|>` The above video shows a robot manipulation trajectory that completes the following task: {instruction}."

We then define the reward for a video prefix $\tau_{1:t}$ to be

$$r_t = \sum_i \log p_\theta(\texttt{inst}_i \mid c(\tau_{1:t}, u, \texttt{inst}_{<i})), \tag{6}$$

where $\texttt{inst}_i$ is the i-th token of the instruction, and u represents the prompt between the video and the instruction. However, we found this alternative formulation to be less effective than the main formulation presented in Section 3. We hypothesize that this is because the model will assign high probability to entities in the instruction if they ever appear in the robot video trajectory. For example, if there is an apple in the video, and the instruction is to peel the apple, then the model will assign high probability to `apple` in the instruction, defeating the purpose of progress estimation. In contrast, our main formulation only requires the model to judge whether the trajectory completes the instruction, which prevents such distraction in probability evaluation.

B Why the `True` Token?

We choose `True` as the affirmative completion token rather than alternatives (e.g., `Yes`) because it is a single token in our evaluated vocabularies and yields the largest, most consistent separation between successful and failed trajectories at the final step. Figure 3 shows the top tokens by absolute difference in mean final-step token probability; `True` exhibits the largest gap.

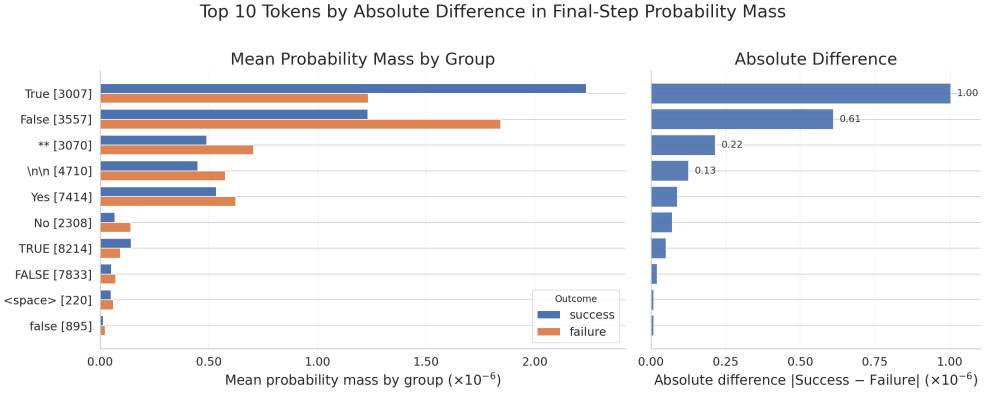


Figure 3: Top 10 tokens by absolute difference in mean final-step token probability between successful and failed trajectories. The affirmative token `True` shows the largest separation, motivating its use as the completion token in `TOPReward`. Left: mean token probability by group; right: absolute difference in mean token probability.

C Success Detection Score

For success detection, we convert `TOPReward` token probabilities into trajectory-level completion scores. Let $r^+_{t_k} = \log p_\theta(\texttt{True} \mid c(\tau_{1:t_k}, x))$ and $r^-_{t_k} = \log p_\theta(\texttt{False} \mid c(\tau_{1:t_k}, x))$ be the completion and non-completion log-probabilities for the k-th sampled prefix, and define the prefix completion margin $m_{t_k} = r^+_{t_k} - r^-_{t_k}$. We first measure how much this margin increases from the beginning to the

end of the trajectory,

$$s_{\text{top}}(\tau, x) = \frac{1}{|\mathcal{L}|} \sum_{k \in \mathcal{L}} m_{t_k} - \frac{1}{|\mathcal{E}|} \sum_{k \in \mathcal{E}} m_{t_k}, \tag{7}$$

where $\mathcal{E}$ and $\mathcal{L}$ denote the first and last three sampled prefixes, respectively. We also score the full video with a direct binary completion query,

$$s_{\text{yn}}(\tau, x) = \log p_\theta(\texttt{Yes} \mid c_{\text{yn}}(\tau, x)) - \log p_\theta(\texttt{No} \mid c_{\text{yn}}(\tau, x)), \tag{8}$$

where c_{yn} is the full-video prompt asking whether the trajectory completed the instruction. For both Qwen3-VL backbones, we use a fixed equal-weight combination of the standardized scores,

$$s_{\text{succ}}(\tau, x) = z(s_{\text{top}}(\tau, x)) + z(s_{\text{yn}}(\tau, x)), \tag{9}$$

where $z(q) = (q - \mu_q)/\sigma_q$ is computed over the evaluated trajectories to put the two scores on comparable scale. This uses no learned classifier or fitted task-specific reward model.

D Dataset-level breakdown

This section provides dataset-level details that complement the aggregate results in Tables 1 and 2. Figure 4 summarizes dataset-level VOC, and Table 5 reports dataset-level VOC for GVL (0-shot) and `TOPReward` (TOPR) for each dataset and model backbone, using the same 33 high-quality OXE datasets as the main table.

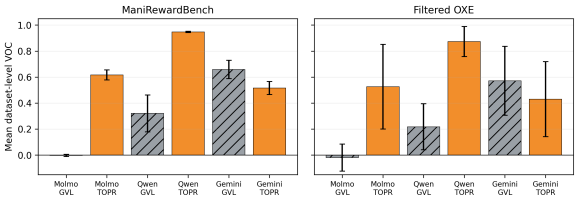


Figure 4: **VOC comparison across datasets.** Mean dataset-level VOC for GVL (0-shot) and `TOPReward` across two evaluation sets: filtered OXE (33 datasets, 20 episodes each) and the `ManiRewardBench` successful progress-evaluation subset (4 datasets, 113 tasks, 497 episodes). Error bars denote standard deviation across datasets within each evaluation set.

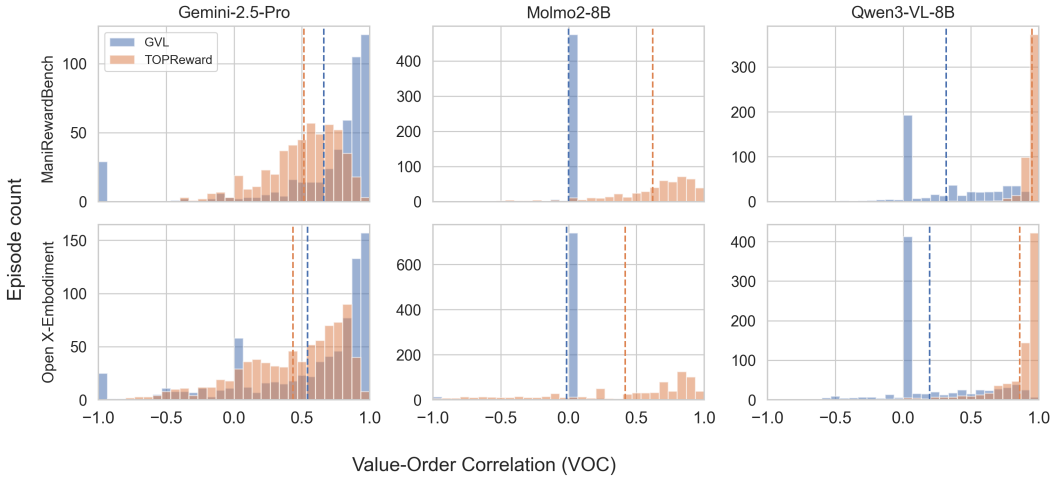


Figure 5: Per-episode VOC distributions, broken down by evaluation set (`ManiRewardBench` vs Open X-Embodiment) and model backbone.

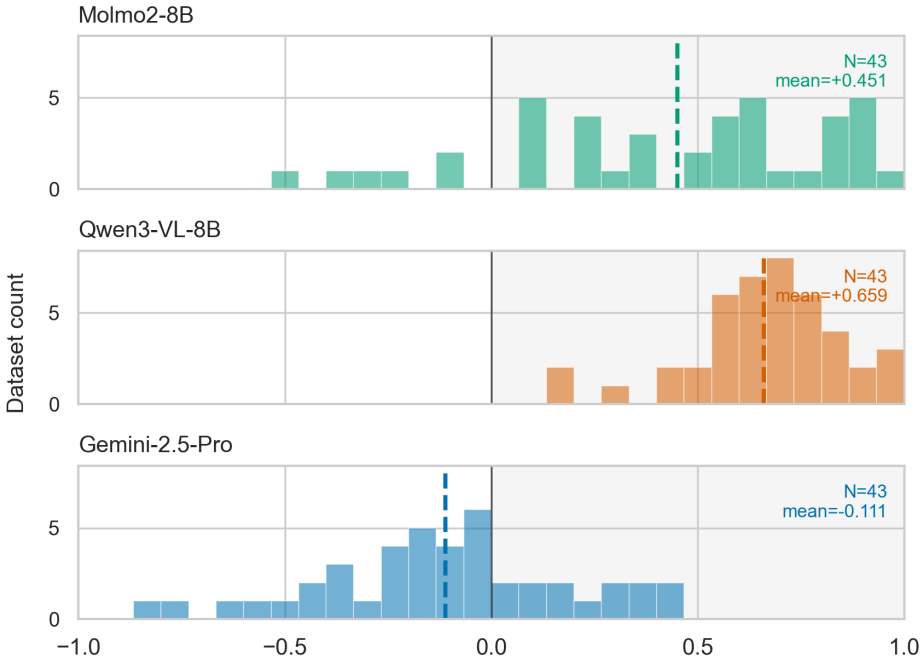


Figure 6: Distribution of dataset-level ΔVOC = VOC(`TOPReward`) − VOC(GVL), shown separately for each model backbone. Positive values indicate `TOPReward` outperforms GVL; the dashed line marks the per-model mean.

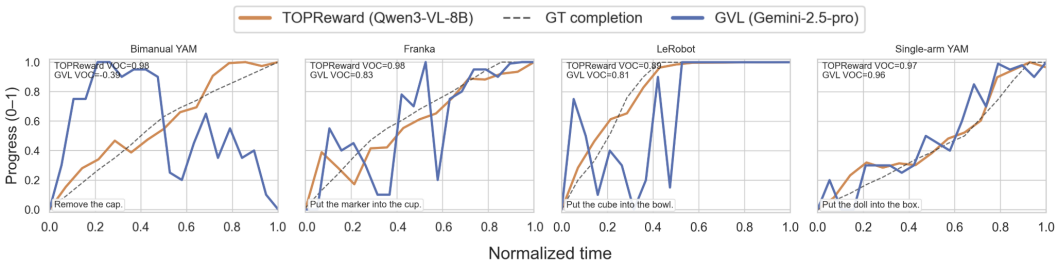


Figure 7: **Progress traces for ManiRewardBench.** Example progress traces predicted by `TOPReward` (orange) compared to stage-aware ground-truth completion (dashed) from `ManiReward Bench`, computed from annotated subtask boundaries. We also overlay Gemini-GVL (blue) on the same episodes when available.

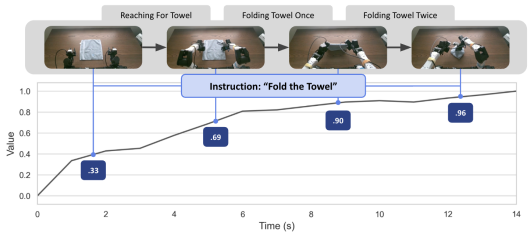


Figure 8: **Qualitative example of "Fold the Towel":** Instruction-conditioned progress estimation on a real trajectory. The curve shows `TOPReward`'s predicted completion value over time, with annotated values at selected frames corresponding to semantic subtasks.

Dataset	Qwen3-VL-8B			Gemini-2.5-Pro			Molmo2-8B		
	GVL	TOPR	Δ	GVL	TOPR	Δ	GVL	TOPR	Δ
ManiRewardBench									
ManiRewardBench_bimanual_yam	0.164	**0.947**	0.783	**0.566**	0.546	-0.021	0.007	**0.565**	0.558
ManiRewardBench_franka	0.242	**0.942**	0.700	**0.695**	0.448	-0.247	0.000	**0.662**	0.662
ManiRewardBench_lerobot	0.332	**0.954**	0.622	**0.620**	0.578	-0.041	-0.001	**0.595**	0.597
ManiRewardBench_single_yam	0.544	**0.945**	0.401	**0.752**	0.488	-0.264	-0.017	**0.642**	0.659
Open X-Embodiment									
aloha_mobile_cabinet	0.199	**0.822**	0.623	**0.814**	0.422	-0.392	0.000	**0.211**	0.211
aloha_mobile_wash_pan	0.127	**0.977**	0.850	**0.856**	0.734	-0.121	0.012	**0.893**	0.880
aloha_mobile_wipe_wine	0.243	**0.961**	0.718	0.699	**0.825**	0.125	0.000	**0.804**	0.804
aloha_static_candy	0.414	**0.963**	0.549	0.697	**0.803**	0.105	0.000	**0.842**	0.842
aloha_static_coffee	0.101	**0.977**	0.876	**0.877**	0.114	-0.764	0.012	**0.400**	0.388
aloha_static_cups_open	0.271	**0.837**	0.565	0.524	**0.765**	0.241	0.000	**0.864**	0.864
aloha_static_pro_pencil	0.000	**0.963**	0.963	**0.457**	-0.029	-0.486	0.000	**0.225**	0.225
aloha_static_screw_driver	0.212	**0.937**	0.725	**0.833**	0.171	-0.662	0.000	**0.925**	0.925
aloha_static_vinh_cup	0.332	**0.916**	0.584	**0.836**	0.723	-0.113	0.000	**0.971**	0.971
aloha_static_vinh_cup_left	-0.057	**0.903**	0.960	0.458	**0.898**	0.441	-0.050	**0.707**	0.757
aloha_static_ziploc_slide	0.301	**0.978**	0.677	**0.845**	0.394	-0.452	0.000	**0.918**	0.918
austin_buds_dataset	0.347	**0.954**	0.607	**0.777**	-0.055	-0.832	0.000	**0.908**	0.908
austin_sirius_dataset	0.242	**0.854**	0.612	**0.704**	0.701	-0.003	0.000	**0.119**	0.119
berkeley_fanuc_manipulation	0.201	**0.866**	0.665	**0.422**	0.333	-0.089	**0.000**	-0.077	-0.077
berkeley_rpt	0.399	**0.983**	0.584	0.303	**0.600**	0.297	0.000	**0.605**	0.605
berkeleymvp	0.240	**0.966**	0.727	**0.744**	0.564	-0.180	0.000	**0.472**	0.472
cmu_franka_exploration_dataset	0.000	**0.626**	0.626	**0.622**	0.272	-0.350	0.000	**0.325**	0.325
dlr_edan_shared_control	0.166	**0.867**	0.700	**0.651**	0.583	-0.068	-0.600	**0.770**	1.370
dlr_sara_grid_clamp	0.000	**0.740**	0.740	**0.253**	-0.166	-0.419	0.000	**0.562**	0.562
jaco_play	0.090	**0.907**	0.818	**0.513**	0.508	-0.005	**0.000**	-0.299	-0.299
kaist_nonprehensile	0.158	**0.914**	0.756	**0.491**	0.349	-0.142	0.000	**0.251**	0.251
nyudoor	0.608	**0.802**	0.194	**0.872**	0.729	-0.142	0.000	**0.650**	0.650
nyufranka	0.232	**0.875**	0.642	**0.772**	0.494	-0.278	0.059	**0.865**	0.806
stanford_hydra_dataset	0.164	**0.973**	0.809	0.379	**0.557**	0.178	0.000	**0.091**	0.091
stanford_kuka_multimodal_dataset	0.122	**0.821**	0.700	-0.390	**-0.055**	0.335	0.000	**0.493**	0.493
stanford_robocook	0.035	**0.443**	0.408	0.329	**0.521**	0.191	0.000	**0.582**	0.582
taco_play	0.015	**0.779**	0.764	**0.050**	0.024	-0.026	0.000	**0.342**	0.342
tokyo_u_lsmo	0.215	**0.977**	0.762	**0.690**	0.501	-0.188	0.000	**0.719**	0.719
ucsd_kitchen_dataset	0.183	**0.718**	0.536	**0.542**	-0.006	-0.547	0.000	**0.228**	0.228
ucsd_pick_and_place_dataset	0.090	**0.819**	0.729	**0.683**	0.520	-0.163	0.000	**0.605**	0.605
utokyo_pr2_opening_fridge	0.485	**0.957**	0.472	0.372	**0.644**	0.272	-0.075	**0.849**	0.924
utokyo_pr2_tabletop_manipulation	0.784	**0.946**	0.162	**0.696**	0.485	-0.211	0.000	**0.121**	0.121
utokyo_xarm_bimanual	0.266	**0.815**	0.549	**0.495**	0.277	-0.218	0.000	**0.386**	0.386

Table 5: Filtered datasets (alphabetical) comparing GVL (0-shot) vs `TOPReward` (TOPR) and their difference, broken down by model backbone. Bold indicates the higher VOC between the two methods for that dataset/model.

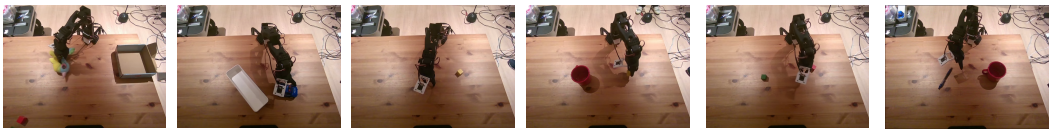
E Additional qualitative results and ablations

E.1 Real-world policy details

For the reward-weighted behavior cloning objective in Equation (5), we use $\beta = 0.2$ and $\delta_{\max} = 2.0$ in Equation (3).

E.2 Chat-template ablation

The Gemini API enforces a chat template, while our preferred open-source inference setting scores the prompt directly without one. To test this factor, we wrap the prompt in Section 3.1 with a chat template and evaluate the probability of the answer being `True`. Table 6 shows that chat formatting substantially reduces VOC for both Qwen3-VL-8B and Molmo2-8B.



(a) Put the doll in box (b) Put toy car in box (c) Pick up the cube (d) Put the cube in cup (e) Stack red cube on green cube (f) Put pen into the cup

Figure 9: The six real-world single-arm SO-100 manipulation tasks used for reward-weighted behavior cloning evaluation.

Figure 10: **Qualitative comparison on "Place doll in box."** The pretrained policy and behavior cloning (BC) both fail, while TOP-RWBC, fine-tuned with reward weights from `TOPReward`, succeeds consistently. Frames are uniformly sampled from evaluation rollouts.

Table 6: **Effect of Chat Template on TOPReward VOC.** Chat template degrades Qwen3-VL-8B performance by nearly 50% and Molmo2-8B by 20%, demonstrating that the logit-based formulation is sensitive to prompt formatting.

	Qwen3-VL-8B		**Molmo2-8B**	
Dataset	Base	+Chat	Base	+Chat
Bimanual YAM	0.947	0.269	0.570	0.408
Franka	0.943	0.528	0.696	0.615
Single-arm YAM	0.946	0.703	0.691	0.546
Mean	**0.945**	**0.500**	**0.652**	**0.523**
Δ	-47.1%		-19.8%	

F Additional details of `ManiRewardBench`

The benchmark includes 130 diverse tasks that capture a wide range of everyday manipulation activities, such as stacking objects, sorting items, and interacting with containers. The dataset is manually collected using *Franka*, *SO-100/101*, *bimanual YAM*, and *single-arm YAM*.

Example challenging tasks in `ManiRewardBench`include:

Multi-step / Reasoning tasks.

- *"Push the puzzles to spell word GO"* (LeRobot) — spatial reasoning combined with sequential multi-object manipulation.
- *"Build a pyramid"* (Bimanual YAM) — multi-step stacking with precise positioning, requiring four distinct subtasks.
- *"Group the cubes by color"* (Bimanual YAM) — requires color perception and categorical reorganization of multiple objects.

- *"Put the cubes of the same colors together"* (Franka) — color-conditional sorting across multiple objects.
- *"Remove the block and stack the green cube on the red cube"* (LeRobot) — obstacle removal followed by color-conditional stacking.
- *"Pack and close the box"* (Franka) — multi-phase task involving packing objects then closing the container.

Fine manipulation / Precise control.

- *"Align the cubes horizontally"* (Bimanual YAM) — fine spatial alignment, corresponding to the longest execution durations in the dataset.
- *"Rotate the banana by 90 degrees"* / *"Rotate the marker by 45 degrees"* (Franka) — precise rotation control with specified angles.
- *"Make the screw points to the glue"* (Bimanual YAM) — precise orientation alignment of two distinct objects.
- *"Pour tea"* (Franka) — requires controlled pouring motion and spatial orientation awareness.

Deformable object handling.

- *"Fold the towel"* / *"Fold towel"* (Franka / Bimanual YAM) — deformable material manipulation requiring careful grasp and fold planning.
- *"Stack one cloth on top of another"* (Single-arm YAM) — soft object stacking with non-rigid geometry.

Abstract / Symbolic tasks.

- *"Press enter and then space key"* (Franka) — keyboard interaction requiring sequential key presses.
- *"Set table"* (Bimanual YAM) — open-ended task requiring understanding of table-setting conventions.

The following table summarizes the statistics of each dataset in `ManiRewardBench`.

Table 7: Summary of `ManiRewardBench` datasets. 6 tasks appear in both the Lerobot and Lerobot failed splits, giving 130 unique tasks across the full benchmark.

Dataset	Episodes	Tasks
Lerobot	150	22
Lerobot failed	156	23
Franka	150	51
Bimanual YAM	97	20
Single-arm YAM	100	20
Total	**653**	**136**

We briefly describe each dataset below:

- **Lerobot Bimanual dataset**: Successful bimanual LeRobot manipulation demos (push, put, remove, stack tasks), 5–10 episodes per task.
- **Lerobot failure dataset**: Mixed failed and successful trajectory examples with the same task types, $\sim$7 episodes per task.
- **Franka dataset**: Franka robot demos with a diverse set of 51 instructions (rotation, cleaning, packing, pick-and-place), mostly 3 episodes per task.

- **Bimanual YAM dataset**: Bimanual YAM manipulation (fold, stack, build, open, etc.), 5 episodes per task.
- **Single-arm YAM dataset**: Single-arm YAM manipulation (put, remove, stack), 5 episodes per task.

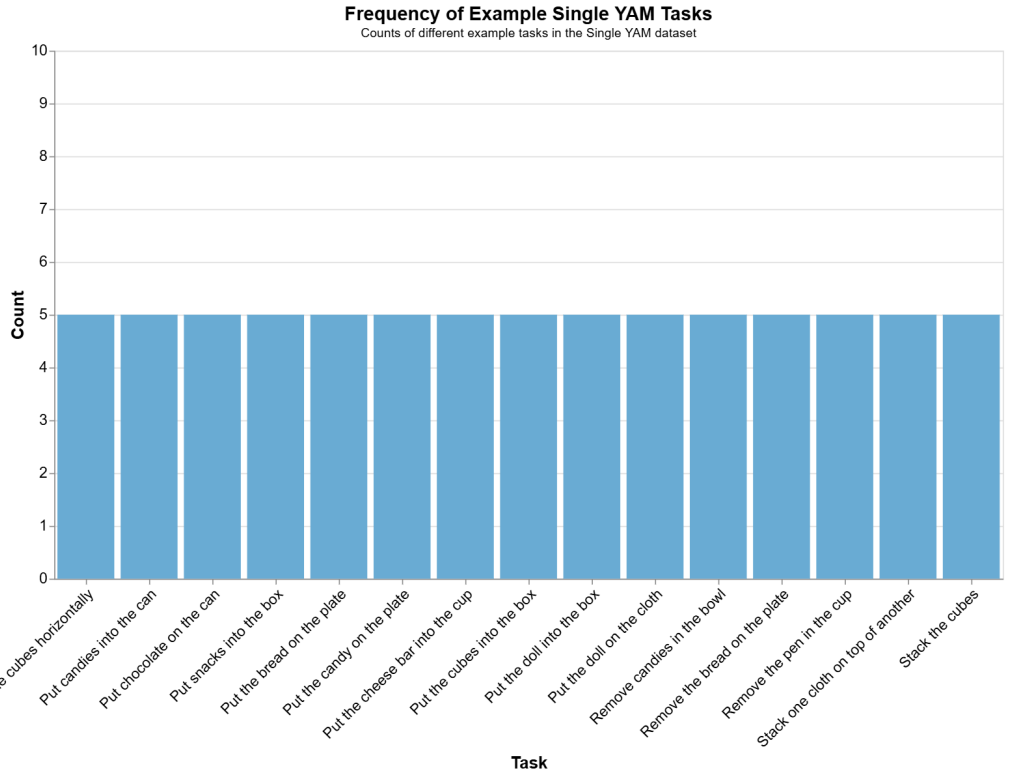


Figure 11: Counts of different example tasks in the single-arm YAM dataset.

F.1 Subtask Annotation

For each task, episodes are manually labeled and segmented into a sequence of predefined subtasks. Each task is associated with an ordered list of subtasks that represent stages of execution (e.g., reaching for an object, grasping it, or placing it). For every subtask, annotators specify a `start_second` (the time in seconds when the subtask begins) and an `end_second` (the time in seconds when it ends). Subtasks are non-overlapping and strictly ordered in time, with each subtask beginning immediately after the previous one ends.

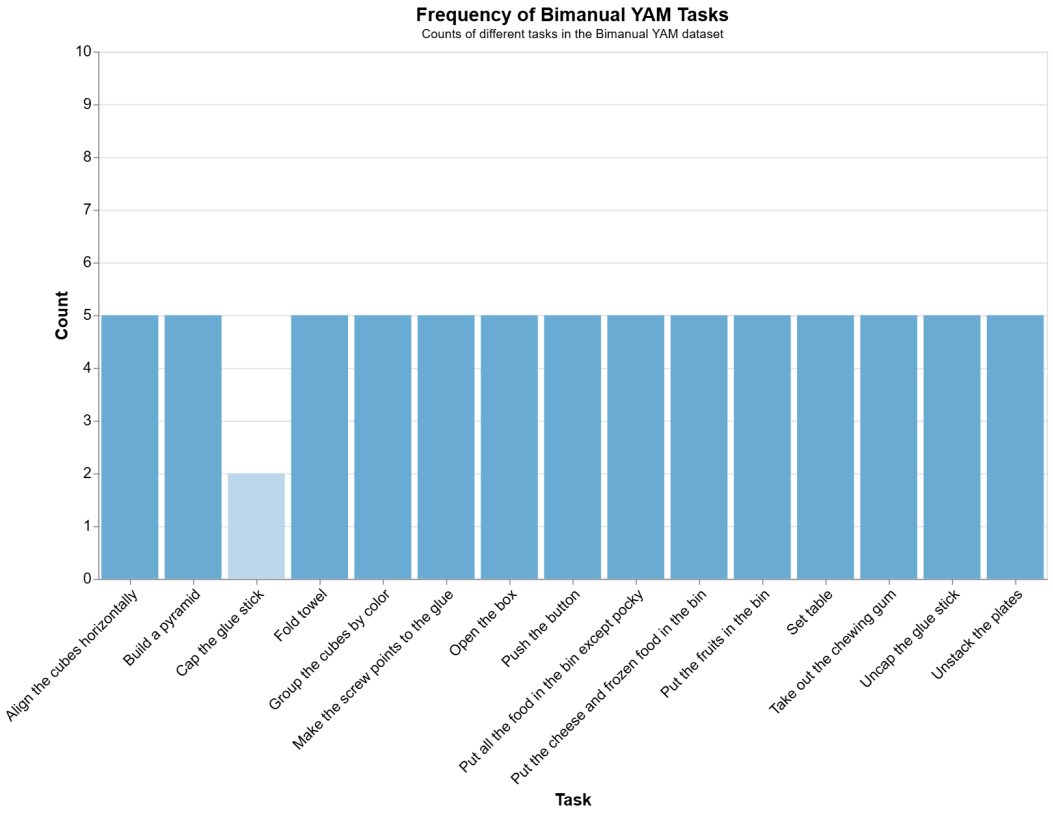


Figure 12: Counts of different example tasks in the bimanual YAM dataset.

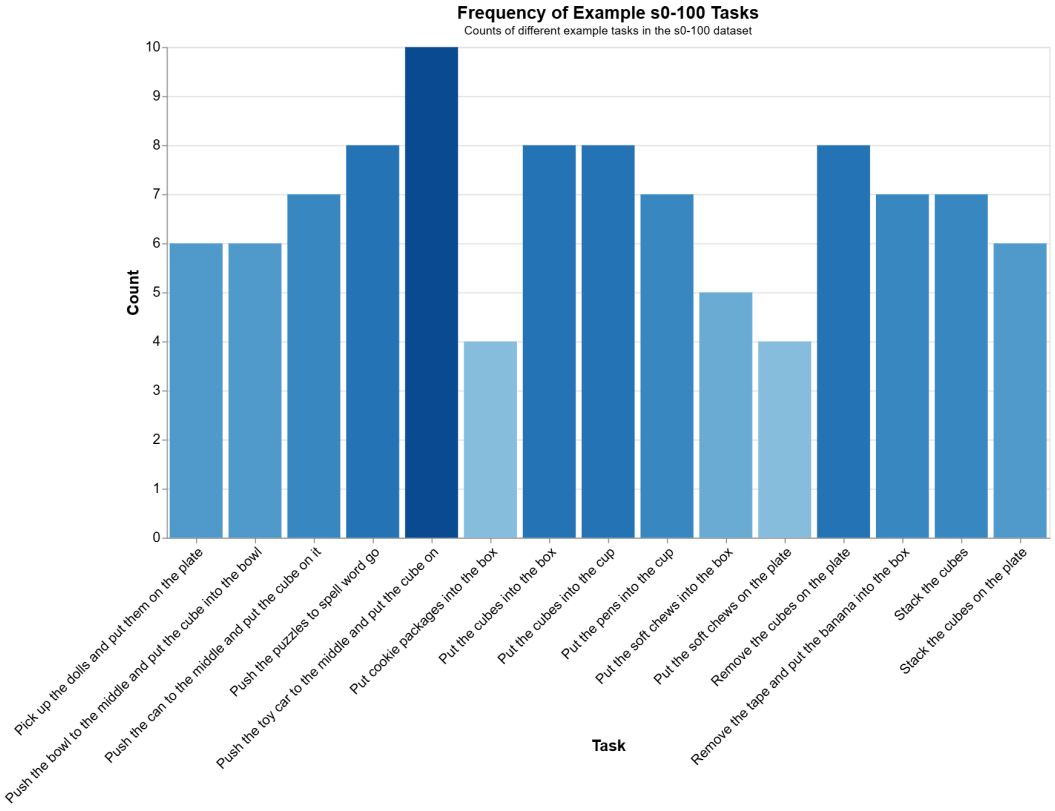


Figure 13: Counts of different example tasks in the SO-100 dataset.

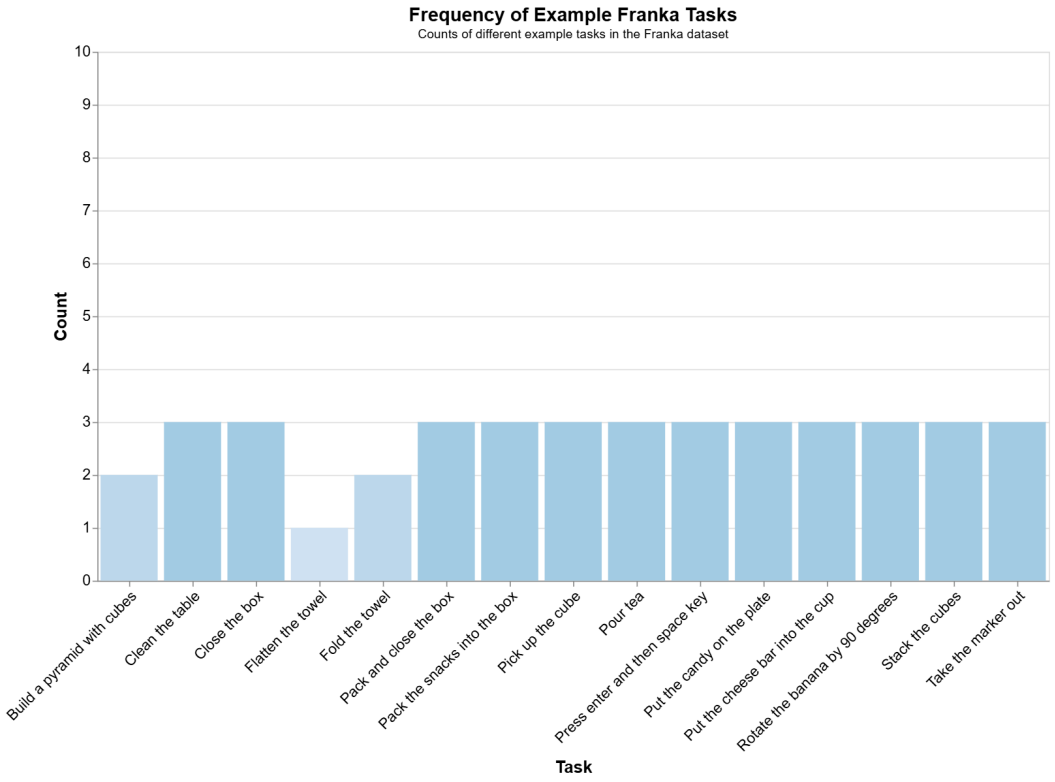


Figure 14: Counts of different example tasks in the Franka dataset.

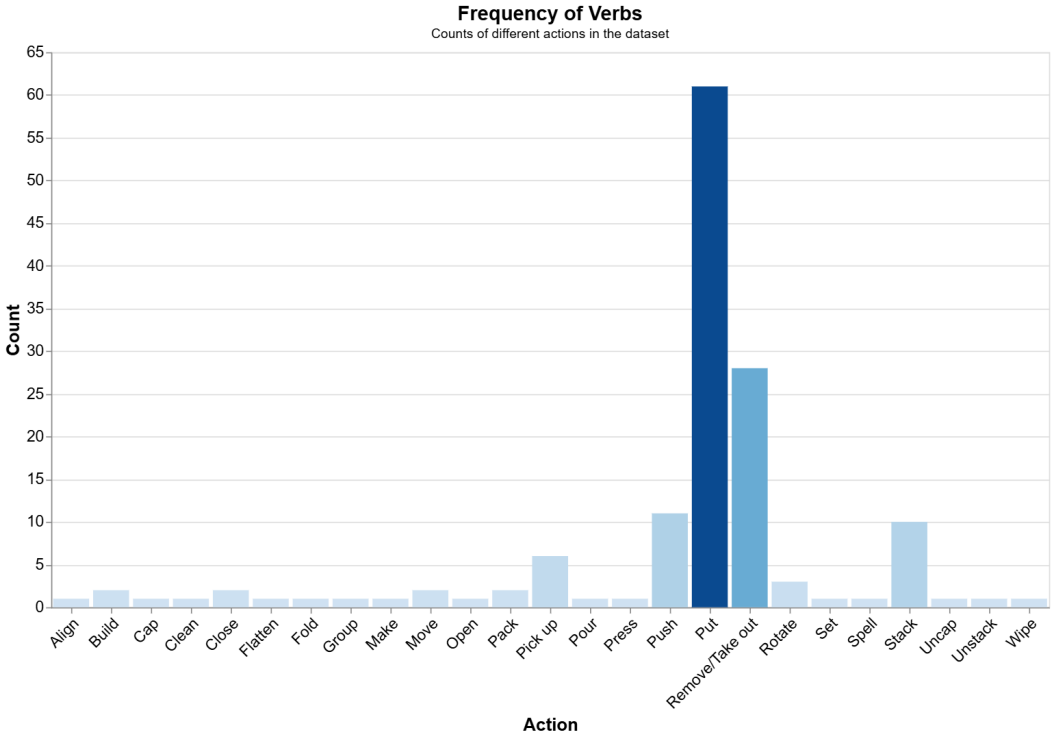


Figure 15: Frequency of verbs

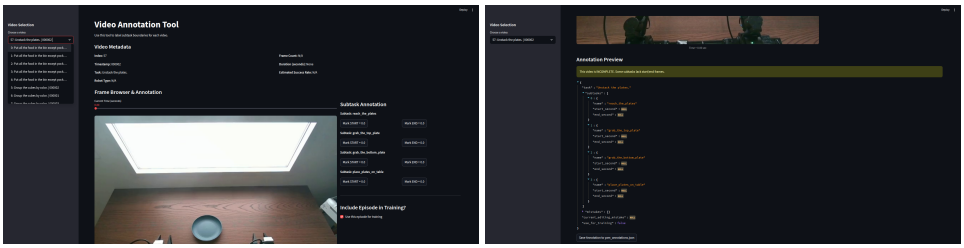
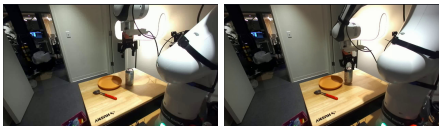
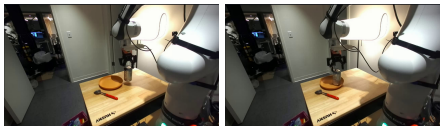


Figure 16: Screenshot of the Annotation Tool.



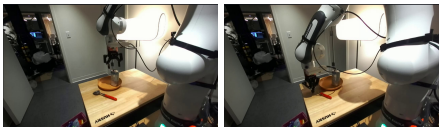
`start_second`: 0.0 `end_second`: 3.9

(a) Grab the can



`start_second`: 4.0 `end_second`: 6.4

(b) Place the can in the plate



`start_second`: 6.5 `end_second`: 9.5

(c) Grab the spoon

`start_second`: 9.6 `end_second`: 11.4

(d) Place the spoon in the plate

Figure 17: Expert demonstration with annotation for the task ”Clean the table”.